NAME:-**Farooqui Shahzeb Ahmed**

ROLL NUMBER:-**SYA-29**

ASSINGMENT:-**01**

AIM:-**To study and implement different sequence data types in Python and perform operations on them.**

Theory:-

Sequence data types are used to store multiple values in an ordered manner. They allow operations like indexing, slicing,iteration, and membership testing. Programs

a) **Strings:-**

```
s = "Hello Python"
print(s[0])

print(s[8:5])

print(s.lower())
```

```
H

hello python
```

b) **Lists:-**

```
# Original corrected list operations
lst = [10, 20, 30, 40]
lst.append(50)
print("Modified List:", lst)
print("Slice [1:3]:", lst[1:3])
```

```
Modified List: [10, 20, 30, 40, 50]
Slice [1:3]: [20, 30]
```

c)**Arrays:-**

```
import array as arr
a = arr.array('i', [1, 2, 3, 4])
a.append(5)
print(a)
```

```
array('i', [1, 2, 3, 4, 5])
```

d)**Tuples:-**

```
t = (1,2,3,4)
print(t[0])
print(t[1:3])
```

```
1
(2, 3)
```

f)**Sets:-**

```
s = {1,2,3,3,4}
print(s)
```

```
{1, 2, 3, 4}
```

g)**Range:-**

```
r =range(1,10,2)
print(list(r))
```

```
[1, 3, 5, 7, 9]
```

Theory Questions & Answers

Q1.** What are sequence data types in Python?**

ANS:- In Python, sequence data types are a category of data structures that store ordered collections of items. These items are arranged in a specific sequence, meaning each element has a defined position (index), and you can access them using that index.

Sequence types share several common features:

- They maintain the order of elements.
- They support indexing (e.g., seq[0]) and slicing (e.g., seq[1:4]).
- They can be iterated over using loops.
- They allow operations like concatenation (+) and repetition (*).

Python provides several built-in sequence data types. The main ones are:

1. List
2. Tuple
3. String
4. Range
5. Bytes
6. Bytearray

```
print("LIST EXAMPLE")
my_list = [1, 2, 3, "apple", 4.5]
print("Original list:", my_list)

my_list[0] = 10
my_list.append("banana")
print("Modified list:", my_list)

print("Slicing:", my_list[1:4])
print()
```

```
print("TUPLE EXAMPLE")
my_tuple = (1, 2, 3, "hello")
print("Tuple:", my_tuple)

print("Access element:", my_tuple[2])
print("Slicing:", my_tuple[1:3])
print()

print("STRING EXAMPLE")
text = "Python"
print("String:", text)

print("First character:", text[0])
print("Slicing:", text[1:4])
print("Concatenation:", text + " Programming")
print()


print("RANGE EXAMPLE")
r = range(1, 10, 2)
print("Range values:")
for i in r:
    print(i, end=" ")
print("\n")

print("BYTES EXAMPLE")
b = bytes([65, 66, 67])
print("Bytes:", b)
print()

print("BYTEARRAY EXAMPLE")
ba = bytearray([65, 66, 67])
print("Original bytearray:", ba)

ba[0] = 68
print("Modified bytearray:", ba)
print()

print("COMMON OPERATIONS")
seq = [10, 20, 30, 40]

print("Indexing:", seq[1])
print("Slicing:", seq[1:3])
print("Concatenation:", seq + [50, 60])
print("Repetition:", seq * 2)
print("Membership test (30 in seq):", 30 in seq)

print("Iteration:")
for item in seq:
    print(item)
```

```
LIST EXAMPLE
Original list: [1, 2, 3, 'apple', 4.5]
Modified list: [10, 2, 3, 'apple', 4.5, 'banana']
Slicing: [2, 3, 'apple']

TUPLE EXAMPLE
Tuple: (1, 2, 3, 'hello')
Access element: 3
Slicing: (2, 3)

STRING EXAMPLE
String: Python
First character: P
```

```
Slicing: yth
Concatenation: Python Programming

RANGE EXAMPLE
Range values:
1 3 5 7 9

BYTES EXAMPLE
Bytes: b'ABC'

BYTEARRAY EXAMPLE
Original bytearray: bytearray(b'ABC')
Modified bytearray: bytearray(b'DBC')

COMMON OPERATIONS
Indexing: 20
Slicing: [20, 30]
Concatenation: [10, 20, 30, 40, 50, 60]
Repetition: [10, 20, 30, 40, 10, 20, 30, 40]
Membership test (30 in seq): True
Iteration:
10
20
30
40
```

Q2. **Difference Between List and Tuple**

ANS:-

Difference Between List and Tuple

Feature	List	Tuple
Mutablity	Mutable (can be changed)	Immutable (cannot be changed)
Syntax	Uses square brackets `[]`	Uses parentheses `()`
Performance	Slower due to overhead of dynamic memory	Faster as they are fixed-size
Use Case	Collections of homogeneous items, data that might change	Collections of heterogeneous items, data that shouldn't c
Operations	`append()`, `insert()`, `remove()`, `sort()`, etc.	Fewer built-in methods (e.g., `count()`, `index()`)
Memory	More memory overhead	Less memory overhead
Readability	Indicates a collection of items	Indicates a fixed sequence of related items
Error Safety	Less error-safe for fixed data	More error-safe for fixed data

```
print("LIST EXAMPLE")
my_list = [1, 2, 3, 4]
print("Original List:", my_list)

my_list[0] = 10
my_list.append(5)
print("Modified List:", my_list)

print()

print("TUPLE EXAMPLE")
my_tuple = (1, 2, 3, 4)
print("Original Tuple:", my_tuple)

try:
    my_tuple[0] = 10
except TypeError as e:
    print("Error:", e)
```

```
print()

import time

start = time.time()
lst = [i for i in range(1000000)]
print("List creation time:", time.time() - start)

start = time.time()
tpl = tuple(i for i in range(1000000))
print("Tuple creation time:", time.time() - start)

print()

import sys

print("List size:", sys.getsizeof(lst))
print("Tuple size:", sys.getsizeof(tpl))
```

Q3. **Arrays vs Lists ANS:-**

Difference Between Python Lists and `array.array` (Arrays)

Feature	List
Data Type	Can store heterogeneous data types (e.g., int, string, float)
Flexibility	Highly flexible, dynamic size
Memory Usage	More memory overhead due to storing type info for each element
Performance	Slower for numerical operations due to type checking overhead
Operations	General-purpose operations (append, insert, remove, sort, etc.)
Creation	`my_list = [1, 'a', 3.0]`
Use Case	General collection of items
Built-in?	Built-in Python data type
Direct Arithmetic	No direct element-wise arithmetic (requires loops or NumPy)
NumPy Arrays	(Note: NumPy arrays are distinct and more powerful for numerical tasks, often preferred over `array.array` for s

When to use arrays?

1. When handling large numeric data
2. When memory efficiency is important

Q4. **Difference Between Sets and Lists ANS:-**

Difference Between Sets and Lists

Feature	List
Order	Ordered (maintains insertion order)
Mutability	Mutable (elements can be changed, added, removed)
Duplicates	Allows duplicate elements
Indexing	Supports indexing (`list[0]`) and slicing (`list[1:3]`)
Syntax	Uses square brackets `[]`
Use Case	Storing collections of items where order matters and duplicates are allowed (e.g., sequence of data, shopping cart ite
Performance	Slower for membership testing on large lists
Operations	`append()`, `insert()`, `remove()`, `sort()`, `len()`, `min()`, `max()`

Feature	List
Heterogeneity	Can store heterogeneous data types

Q5. **How Dictionaries Work Internally**

ANS:- Python dictionaries work internally using a hash table, which allows them to store data as key–value pairs and access it very quickly. When you insert a key, Python applies a hash function to that key to generate a unique integer (called a hash value). This hash value determines the index where the value will be stored in an internal array. When you try to access the value later, Python hashes the key again and directly jumps to the correct location, making lookup very fast (average time complexity O(1)).

If two keys produce the same index (called a collision), Python handles it using techniques like probing to find another empty slot. Also, only immutable types (like strings, numbers, or tuples) can be used as keys because their hash value must remain constant. Dictionaries automatically resize when they grow, ensuring performance stays efficient, and in modern Python versions, they also maintain the insertion order.

```
data = {"name": "Alice", "age": 25, "city": "Pune"}

print("Name:", data["name"])
print("Age:", data["age"])

print("Hash of 'name':", hash("name"))
print("Hash of 'age':", hash("age"))

data["country"] = "India"
print("Updated Dictionary:", data)

valid_dict = {(1, 2): "Tuple Key"}  # valid key
print("Valid Key Example:", valid_dict)
```

```
Name: Alice
Age: 25
Hash of 'name': -7200111249609121374
Hash of 'age': 8546354186312689758
Updated Dictionary: {'name': 'Alice', 'age': 25, 'city': 'Pune', 'country': 'India'}
Valid Key Example: {(1, 2): 'Tuple Key'}
```

Conclusion

In this practical, we studied and implemented different sequence data types in Python, including lists, tuples, strings, range, bytes, and bytearray. We explored their characteristics such as ordered storage, indexing, slicing, and iteration, and performed various operations like concatenation, repetition, and membership testing. We also understood the key difference between mutable and immutable sequence types, which is important when choosing the appropriate data structure for a task.

Overall, this practical helped build a strong foundation in handling sequential data in Python. These concepts are essential in data science for tasks like data manipulation, processing, and analysis, making sequence data types a fundamental part of efficient programming.